pe_row — Microarchitecture Specification

Document status: DRAFT — for implementation **IP block:** Systolic Array for AI Accelerators — Week 3 **Module under spec:** `pe_row` **Author:** (architect) | **Implementer:** Jayrajsinh Chauhan | **Reviewer:** —

---

1. Purpose

`pe_row` composes `NUM_PE` instances of the already-verified `processing_element` block into a single unit that computes a sliding-window weighted sum over a streamed activation sequence. This is the Week 3 milestone: prove that multiple weight-stationary PEs compose correctly before scaling to a 2D array.

This document specifies **what** the block must do and **the exact contract** it must honor. It does not specify internal implementation — that is the RTL engineer's design decision, subject to review.

---

2. Interface Specification

Signal	Dir	Width	Description
`clk`	in	1	System clock
`rst_n`	in	1	Active-low async reset
`load_compute_sel`	in	1	`0` = load-weight mode, `1` = compute mode (same semantics as `processing_element`)
`activation_in`	in	`DATA_WIDTH`	Activation value, **applies to the whole row this cycle** (see §4.2)
`weight_in_flat`	in	`NUM_PE * DATA_WIDTH`	Flattened bus carrying one stationary weight per PE. Bit-slice convention is an implementation decision — document your convention in your own header comment and in your testbench
`psum_out`	out	`ACC_WIDTH`	Accumulated result of the full row
`overflow_flag`	out	1	Asserted if **any** PE in the row reports overflow this cycle

---

3. Parameters

Parameter	Default	Constraint
`DATA_WIDTH`	8	Must match the instantiated `processing_element`'s `DATA_WIDTH`
`ACC_WIDTH`	20	Must match the instantiated `processing_element`'s `ACC_WIDTH`
`NUM_PE`	3	⩾ 1. Design must not hardcode a specific value of `NUM_PE` anywhere in the logic — use it generatively

4. Functional Description

4.1 Reuse constraint

`pe_row` **must be built by instantiating** `processing_element` **— not by re-deriving the MAC logic.** The single PE is already verified (5/5 tests, race-condition-fixed); re-implementing its logic inside `pe_row` would throw away that verification and duplicate a bug surface. If you find yourself writing a multiply or an accumulator directly inside `pe_row`, stop — that logic belongs in `processing_element`, already exists there, and should be reused via instantiation.

4.2 Activation distribution — this is the critical design decision, read carefully

Every PE in the row must observe the **same** `activation_in` value on the **same** clock cycle. Do **not** forward one PE's registered output into the next PE's activation input — that produces a different (and incorrect, for this spec's intended behavior) mathematical result. Think through, on paper, what value each PE's `activation_in` holds at a given cycle under each wiring choice before you commit to one. This is exactly the kind of decision a design review would catch if gotten wrong — work it out yourself rather than guessing.

4.3 Partial sum propagation

The partial sum must accumulate progressively as it moves through the row: PE 0 contributes the first term, and each subsequent PE adds its own weighted term to what it receives from the previous PE. The **first** PE in the chain has no upstream partial sum to receive — decide what value it should start from, and justify it (hint: what value added to anything leaves that thing unchanged?).

4.4 Functional intent (black-box behavior)

Treat `pe_row` as a black box for a moment — ignore how it's built internally. Given stationary weights `w[0..NUM_PE-1]` and a streamed sequence `x[1], x[2], x[3], ...` (one value per cycle, in compute mode), the block should, after a warm-up period, produce on `psum_out`:

```
y[i] = w[0]·x[i] + w[1]·x[i+1] + w[2]·x[i+2] + ... + w[NUM_PE-1]·x[i+NUM_PE-1]
```

a new such value each cycle once the pipeline is full. **Your job**: work out, from the interface and §4.1–4.3, exactly which cycle the first valid `y[1]` appears on, and what `psum_out` holds on every cycle before and after that — including during the load-weight phase. Write this down as a cycle-by-cycle table before writing RTL. This is standard practice — a timing table you can defend is worth more than RTL you can't explain.

4.5 Overflow

`overflow_flag` must reflect whether **any** PE in the row is currently reporting its own overflow condition. Decide the combining logic (there's a standard bitwise operation for "any of N bits is set") and be able to state why it's correct for `NUM_PE` PEs, not just for 3.

---

5. Design Constraints

- Must be parameterizable by `NUM_PE` — a reviewer should be able to change `NUM_PE` to 5 and expect correct behavior without editing the module body.
- Must not introduce any new arithmetic beyond what `processing_element` already provides.
- Bit-slicing convention for `weight_in_flat` is your choice — document it in a comment at the top of the file, and make sure your testbench's weight-loading matches it exactly (a mismatch here is a classic self-inflicted bug — the DUT and testbench silently disagreeing on convention).

---

6. Verification Plan (write this before, or alongside, the RTL)

A real DVP lists scenarios before implementation exists, so verification isn't an afterthought. Write directed test cases covering at minimum:

1. **Basic correctness** — load a small set of weights, stream a short known sequence, verify `psum_out` against hand-calculated expected values (compute these by hand first, the same way you validated the single PE's ACC_WIDTH math)
2. **Weight reload** — confirm loading new weights mid-sequence correctly changes subsequent outputs, and does not corrupt in-flight partial sums
3. **Overflow** — force at least one PE's contribution to overflow, confirm `overflow_flag` asserts at the correct cycle
4. **Reset behavior** — assert `rst_n` mid-stream, confirm the row returns to a known, documented state
5. **Timing check against your own §4.4 table** — this is the one that catches a design bug at the RTL/testbench boundary, not just an obviously-wrong final number

For each test, state the **expected value before running the simulator** — same discipline that caught the race condition in the single-PE testbench. A test whose expected value you

compute *after* seeing simulator output isn't verifying anything.

---

7. Definition of Done

- [ ] `verilator --lint-only -Wall` clean (or every warning explicitly understood and justified, not blanket-suppressed)
- [ ] All directed tests in §6 pass, with self-checking pass/fail output (not manual waveform eyeballing)
- [ ] Waveform reviewed and visually confirms the §4.4 timing table
- [ ] Design decisions from §4.2–4.5 documented in a comment block at the top of `pe_row.v` — a reviewer (or future you) should understand *why* the wiring is what it is without re-deriving it

---

8. Design Decisions Log (fill this in as you go)

(This is standard practice — real IP blocks carry a decisions log so reviewers and future engineers understand not just what was built, but why. Fill in your own reasoning as you make each call.)

Decision	Your choice	Rationale
Activation distribution (§4.2)		
First PE's incoming psum (§4.3)		
`weight_in_flat` bit-slice convention (§5)		
Overflow combining logic (§4.5)		